The JDBC(tm) API Version 1.10

October 21, 1996

Part 2 - Classes and Exceptions

This document contains a paper copy of the JDBC API online documentation that is distributed with the JDBC package and is also available on http://splash.javasoft.com/jdbc.

It takes the place of the source code comments that were originally included as part of the JDBC specification.

---

package java.sql

Interface Index

- CallableStatement
- Connection
- DatabaseMetaData
- Driver
- PreparedStatement
- ResultSet
- ResultSetMetaData
- Statement

Class Index

- Date
- DriverManager
- DriverPropertyInfo
- Time
- Timestamp
- Types

Exception Index

- DataTruncation
- SQLException
- SQLWarning

Class java.sql.Date

```
java.lang.Object
   |
   +----java.util.Date
           |
           +----java.sql.Date
```

---

public class **Date**

extends Date

This class is a thin wrapper around java.util.Date that allows JDBC to identify this as a SQL DATE value. It adds formatting and parsing operations to support the JDBC escape syntax for date values.

Note: Although we recommend against it, nothing prevents the use of the full facilities of the underlying java.util.Date class. For instance, the setHours method could be used and this would result in a java.sql.Date value that would not compare properly with other Date values.

Constructor Index

Method Index

Constructors

Date

```
public Date(int year,
            int month,
            int day)
```

Construct a Date
Parameters:
year - year-1900
month - 0 to 11
day - 1 to 31

Date

```
public Date(long date)
```

Construct a Date using a milliseconds time value
Parameters:
date - milliseconds since January 1, 1970, 00:00:00 GMT

Methods

- **valueOf**

```
public static Date valueOf(String s)
```

Convert a string in JDBC date escape format to a Date value
Parameters:
s - date in format "yyyy-mm-dd"
Returns:
corresponding Date

- **toString**

```
public String toString()
```

Format a date in JDBC date escape format
Returns:
a String in yyyy-mm-dd format
Overrides:
toString in class Date

Class java.sql.DriverManager

```
java.lang.Object
   |
   +----java.sql.DriverManager
```

---

public class **DriverManager**
extends Object

The DriverManager provides a basic service for managing a set of JDBC drivers.

As part of its initialization, the DriverManager class will attempt to load the driver classes referenced in the "jdbc.drivers" system property. This allows a user to customize the JDBC Drivers used by their applications. For example in your ~/.hotjava/properties file you might specify: `jdbc.drivers=foo.bah.Driver:wombat.sql.Driver:bad.taste.ourDriver` A program can also explicitly load JDBC drivers at any time. For example, the my.sql.Driver is loaded with the following statement: `Class.forName("my.sql.Driver");`

When getConnection is called the DriverManager will attempt to locate a suitable driver from amongst those loaded at initialization and those loaded explicitly using the same classloader as the current applet or application.

See Also:
Driver, Connection

Method Index

- **deregisterDriver**(Driver)
  Drop a Driver from the DriverManager's list.
- **getConnection**(String)
  Attempt to establish a connection to the given database URL.
- **getConnection**(String, Properties)
  Attempt to establish a connection to the given database URL.
- **getConnection**(String, String, String)
  Attempt to establish a connection to the given database URL.
- **getDriver**(String)
  Attempt to locate a driver that understands the given URL.
- **getDrivers**()
  Return an Enumeration of all the currently loaded JDBC drivers which the current caller has access to.
- **getLoginTimeout**()
  Get the maximum time in seconds that all drivers can wait when attempting to log in to a database.
- **getLogStream**()
  Get the logging/tracing PrintStream that is used by the DriverManager and all drivers.
- **println**(String)
  Print a message to the current JDBC log stream
- **registerDriver**(Driver)
  A newly loaded driver class should call registerDriver to make itself known to the DriverManager.
- **setLoginTimeout**(int)
  Set the maximum time in seconds that all drivers can wait when attempting to log in to a database.
- **setLogStream**(PrintStream)
  Set the logging/tracing PrintStream that is used by the DriverManager and all drivers.

Methods

- **getConnection**

```
public static synchronized Connection getConnection(String url,
                                                    Properties info) throws SQLExc
```

Attempt to establish a connection to the given database URL. The DriverManager attempts to select an appropriate driver from the set of registered JDBC drivers.

Parameters:

url - a database url of the form jdbc:*subprotocol*:*subname*
info - a list of arbitrary string tag/value pairs as connection arguments; normally at least a

"user" and "password" property should be included

Returns:

a Connection to the URL

- **getConnection**

```
public static synchronized Connection getConnection(String url,
                                                    String user,
                                                    String password) throws SQLExc
```

Attempt to establish a connection to the given database URL. The DriverManager attempts to select an appropriate driver from the set of registered JDBC drivers.

Parameters:

url - a database url of the form jdbc:*subprotocol*:*subname*

user - the database user on whose behalf the Connection is being made

password - the user's password

Returns:

a Connection to the URL

- **getConnection**

```
public static synchronized Connection getConnection(String url) throws SQLExceptio
```

Attempt to establish a connection to the given database URL. The DriverManager attempts to select an appropriate driver from the set of registered JDBC drivers.

Parameters:

url - a database url of the form jdbc:*subprotocol*:*subname*

Returns:

a Connection to the URL

- **getDriver**

```
public static Driver getDriver(String url) throws SQLException
```

Attempt to locate a driver that understands the given URL. The DriverManager attempts to select an appropriate driver from the set of registered JDBC drivers.

Parameters:

url - a database url of the form jdbc:*subprotocol*:*subname*

Returns:

a Driver that can connect to the URL

- **registerDriver**

```
public static synchronized void registerDriver(Driver driver) throws SQLException
```

A newly loaded driver class should call registerDriver to make itself known to the DriverManager.

Parameters:

driver - the new JDBC Driver

- **deregisterDriver**

```
public static void deregisterDriver(Driver driver) throws SQLException
```

Drop a Driver from the DriverManager's list. Applets can only deregister Drivers from their own classloader.

Parameters:

driver - the JDBC Driver to drop

- **getDrivers**

```
public static Enumeration getDrivers()
```

Return an Enumeration of all the currently loaded JDBC drivers which the current caller has access to.

Note: The classname of a driver can be found using `d.getClass().getName()`

Returns:

the list of JDBC Drivers loaded by the caller's class loader

- **setLoginTimeout**

```
public static void setLoginTimeout(int seconds)
```

Set the maximum time in seconds that all drivers can wait when attempting to log in to a database.

Parameters:

seconds - the driver login time limit

- **getLoginTimeout**

```
public static int getLoginTimeout()
```

Get the maximum time in seconds that all drivers can wait when attempting to log in to a database.

Returns:

the driver login time limit

- **setLogStream**

```
public static void setLogStream(PrintStream out)
```

Set the logging/tracing PrintStream that is used by the DriverManager and all drivers.

Parameters:

out - the new logging/tracing PrintStream; to disable, set to null

- **getLogStream**

```
public static PrintStream getLogStream()
```

Get the logging/tracing PrintStream that is used by the DriverManager and all drivers.

Returns:

the logging/tracing PrintStream; if disabled, is null

- **println**

```
public static void println(String message)
```

  Print a message to the current JDBC log stream
  Parameters:
    message - a log or tracing message

Class java.sql.DriverPropertyInfo

```
java.lang.Object
   |
   +----java.sql.DriverPropertyInfo
```

---

public class **DriverPropertyInfo**
extends Object

The DriverPropertyInfo class is only of interest to advanced programmers who need to interact with a Driver via getDriverProperties to discover and supply properties for connections.

Variable Index

- **choices**
  If the value may be selected from a particular set of values, then this is an array of the possible values.
- **description**
  A brief description of the property.
- **name**
  The name of the property.
- **required**
  "required" is true if a value must be supplied for this property during Driver.connect.
- **value**
  "value" specifies the current value of the property, based on a combination of the information supplied to getPropertyInfo, the Java environment, and driver supplied default values.

Constructor Index

- **DriverPropertyInfo**(String, String)
  Constructor a DriverPropertyInfo with a name and value; other members default to their initial values.

Variables

- **name**

```
public String name
```

  The name of the property.

- **description**

```
public String description
```

  A brief description of the property. This may be null.

- **required**

```
public boolean required
```

  "required" is true if a value must be supplied for this property during Driver.connect. Otherwise the property is optional.

- **value**

```
public String value
```

  "value" specifies the current value of the property, based on a combination of the information supplied to getPropertyInfo, the Java environment, and driver supplied default values. This may be null if no value is known.

- **choices**

```
public String choices[]
```

  If the value may be selected from a particular set of values, then this is an array of the possible values. Otherwise it should be null.

Constructors

- **DriverPropertyInfo**

```
public DriverPropertyInfo(String name,
                          String value)
```

  Constructor a DriverPropertyInfo with a name and value; other members default to their initial values.

  Parameters:

  name - the name of the property
  value - the current value, which may be null

Class java.sql.Time

```
java.lang.Object
   |
   +----java.util.Date
           |
           +----java.sql.Time
```

public class **Time**
extends Date

This class is a thin wrapper around java.util.Date that allows JDBC to identify this as a SQL TIME value. It adds formatting and parsing operations to support the JDBC escape syntax for time values.

Note: Although we recommend against it, nothing prevents the use of the full facilities of the underlying java.util.Date class. For instance, the setMonth method could be used and this would result in a java.sql.Time value that would not compare properly with other Time values.

Constructor Index

Method Index

Constructors

- **Time**

```
public Time(int hour,
            int minute,
            int second)
```

Construct a Time Object
Parameters:
hour - 0 to 23
minute - 0 to 59
second - 0 to 59

- **Time**

```
public Time(long time)
```

Construct a Time using a milliseconds time value
Parameters:
time - milliseconds since January 1, 1970, 00:00:00 GMT

Methods

- **valueOf**

```
public static Time valueOf(String s)
```

Convert a string in JDBC time escape format to a Time value
Parameters:
s - time in format "hh:mm:ss"
Returns:
corresponding Time

- **toString**

```
public String toString()
```

Format a time in JDBC date escape format
Returns:
a String in hh:mm:ss format
Overrides:
toString in class Date

Class java.sql.Timestamp

```
java.lang.Object
   |
   +----java.util.Date
           |
           +----java.sql.Timestamp
```

---

public class **Timestamp**
extends Date

This class is a thin wrapper around java.util.Date that allows JDBC to identify this as a SQL TIMESTAMP value. It adds the ability to hold the SQL TIMESTAMP nanos value and provides formatting and parsing operations to support the JDBC escape syntax for timestamp values.

Note: This type is a composite of a java.util.Date and a separate nanos value. Only integral seconds are stored in the java.util.Date component. The fractional seconds - the nanos - are separate. The getTime method will only return integral seconds. If a time value that includes the fractional seconds is desired you must convert nanos to milliseconds (nanos/1000000) and add this to the getTime value. Also note that the hashcode() method uses the underlying java.util.Data implementation and therefore does not include nanos in its computation.

Constructor Index

- **Timestamp**(int, int, int, int, int, int, int)
  Construct a Timestamp Object
- **Timestamp**(long)
  Construct a Timestamp using a milliseconds time value.

Method Index

- **after**(Timestamp)
  Is this timestamp later than the timestamp argument?
- **before**(Timestamp)
  Is this timestamp earlier than the timestamp argument?
- **equals**(Timestamp)
  Test Timestamp values for equality
- **getNanos**()
  Get the Timestamp's nanos value
- **setNanos**(int)
  Set the Timestamp's nanos value
- **toString**()
  Format a timestamp in JDBC timestamp escape format
- **valueOf**(String)
  Convert a string in JDBC timestamp escape format to a Timestamp value

Constructors

- **Timestamp**

```
public Timestamp(int year,
```

```
                int month,
                int date,
                int hour,
                int minute,
                int second,
                int nano)
```

Construct a Timestamp Object
Parameters:
year - year-1900
month - 0 to 11
day - 1 to 31
hour - 0 to 23
minute - 0 to 59
second - 0 to 59
nano - 0 to 999,999,999

- **Timestamp**

```
public Timestamp(long time)
```

Construct a Timestamp using a milliseconds time value. The integral seconds are stored in the underlying date value; the fractional seconds are stored in the nanos value.
Parameters:
time - milliseconds since January 1, 1970, 00:00:00 GMT

- **valueOf**

```
public static Timestamp valueOf(String s)
```

Convert a string in JDBC timestamp escape format to a Timestamp value
Parameters:
s - timestamp in format "yyyy-mm-dd hh:mm:ss.fffffffff"
Returns:
corresponding Timestamp

- **toString**

```
public String toString()
```

Format a timestamp in JDBC timestamp escape format
Returns:
a String in yyyy-mm-dd hh:mm:ss.fffffffff format
Overrides:
toString in class Date

- **getNanos**

```
public int getNanos()
```

Get the Timestamp's nanos value
Returns:
the Timestamp's fractional seconds component

- **setNanos**

```
public void setNanos(int n)
```

Set the Timestamp's nanos value
Parameters:
n - the new fractional seconds component

- **equals**

```
public boolean equals(Timestamp ts)
```

Test Timestamp values for equality
Parameters:
ts - the Timestamp value to compare with

- **before**

```
public boolean before(Timestamp ts)
```

Is this timestamp earlier than the timestamp argument?
Parameters:
ts - the Timestamp value to compare with

- **after**

```
public boolean after(Timestamp ts)
```

Is this timestamp later than the timestamp argument?
Parameters:
ts - the Timestamp value to compare with

Class java.sql.Types

```
java.lang.Object
   |
   +----java.sql.Types
```

---

public class **Types**
extends Object

This class defines constants that are used to identify SQL types. The actual type constant values are equivalent to those in XOPEN.

Variable Index

Variables

- **BIT**

```
public final static int BIT
```

- **TINYINT**

```
public final static int TINYINT
```

- **SMALLINT**

```
public final static int SMALLINT
```

- **INTEGER**

public final static int INTEGER

- **BIGINT**

  public final static int BIGINT

- **FLOAT**

  public final static int FLOAT

- **REAL**

  public final static int REAL

- **DOUBLE**

  public final static int DOUBLE

- **NUMERIC**

  public final static int NUMERIC

- **DECIMAL**

  public final static int DECIMAL

- **CHAR**

  public final static int CHAR

- **VARCHAR**

  public final static int VARCHAR

- **LONGVARCHAR**

  public final static int LONGVARCHAR

- **DATE**

  public final static int DATE

- **TIME**

  public final static int TIME

- **TIMESTAMP**

  public final static int TIMESTAMP

- **BINARY**

  public final static int BINARY

- **VARBINARY**

  ```
  public final static int VARBINARY
  ```

- **LONGVARBINARY**

  ```
  public final static int LONGVARBINARY
  ```

- **NULL**

  ```
  public final static int NULL
  ```

- **OTHER**

  ```
  public final static int OTHER
  ```

  OTHER indicates that the SQL type is database specific and gets mapped to a Java object which can be accessed via getObject and setObject.

Class java.sql.DataTruncation

```
java.lang.Object
   |
   +----java.lang.Throwable
           |
           +----java.lang.Exception
                   |
                   +----java.sql.SQLException
                           |
                           +----java.sql.SQLWarning
                                   |
                                   +----java.sql.DataTruncation
```

---

public class **DataTruncation**
extends SQLWarning

When JDBC unexpectedly truncates a data value, it reports a DataTruncation warning (on reads) or throws a DataTruncation exception (on writes).

The SQLstate for a DataTruncation is "01004".

Constructor Index

- **DataTruncation**(int, boolean, boolean, int, int)

Create a DataTruncation object.

Method Index

- **getDataSize**()
  Get the number of bytes of data that should have been transferred.
- **getIndex**()
  Get the index of the column or parameter that was truncated.
- **getParameter**()
  Is this a truncated parameter value?
- **getRead**()
  Was this a read truncation?
- **getTransferSize**()
  Get the number of bytes of data actually transferred.

Constructors

DataTruncation

```
public DataTruncation(int index,
                      boolean parameter,
                      boolean read,
                      int dataSize,
                      int transferSize)
```

Create a DataTruncation object. The SQLState is initialized to 01004, the reason is set to "Data truncation" and the vendorCode is set to the SQLException default.

Parameters:

index - The index of the parameter or column value
parameter - true if a parameter value was truncated
read - true if a read was truncated
dataSize - the original size of the data
transferSize - the size after truncation

Methods

getIndex

```
public int getIndex()
```

Get the index of the column or parameter that was truncated.

This may be -1 if the column or parameter index is unknown, in which case the "parameter" and "read" fields should be ignored.

Returns:

the index of the truncated paramter or column value.

- **getParameter**

```
public boolean getParameter()
```

  Is this a truncated parameter value?
  Returns:
    True if the value was a parameter; false if it was a column value.

- **getRead**

```
public boolean getRead()
```

  Was this a read truncation?
  Returns:
    True if the value was truncated when read from the database; false if the data was truncated on a write.

- **getDataSize**

```
public int getDataSize()
```

  Get the number of bytes of data that should have been transferred. This number may be approximate if data conversions were being performed. The value may be "-1" if the size is unknown.
  Returns:
    the number of bytes of data that should have been transferred

- **getTransferSize**

```
public int getTransferSize()
```

  Get the number of bytes of data actually transferred. The value may be "-1" if the size is unknown.
  Returns:
    the number of bytes of data actually transferred

Class java.sql.SQLException

```
java.lang.Object
   |
   +----java.lang.Throwable
           |
           +----java.lang.Exception
                   |
                   +----java.sql.SQLException
```

---

public class **SQLException**
extends Exception

The SQLException class provides information on a database access error.

Each SQLException provides several kinds of information:

- a string describing the error. This is used as the Java Exception message, and is available via the getMesage() method
- A "SQLstate" string which follows the XOPEN SQLstate conventions. The values of the SQLState string as described in the XOPEN SQL spec.
- An integer error code that is vendor specific. Normally this will be the actual error code returned by the underlying database.
- A chain to a next Exception. This can be used to provided additional error information.

Constructor Index

- **SQLException**()
  Construct an SQLException; reason defaults to null, SQLState defaults to null and vendorCode defaults to 0.
- **SQLException**(String)
  Construct an SQLException with a reason; SQLState defaults to null and vendorCode defaults to 0.
- **SQLException**(String, String)
  Construct an SQLException with a reason and SQLState; vendorCode defaults to 0.
- **SQLException**(String, String, int)
  Construct a fully-specified SQLException

Method Index

- **getErrorCode**()
  Get the vendor specific exception code
- **getNextException**()
  Get the exception chained to this one.
- **getSQLState**()
  Get the SQLState
- **setNextException**(SQLException)
  Add an SQLException to the end of the chain.

Constructors

- **SQLException**

```
public SQLException(String reason,
                    String SQLState,
                    int vendorCode)
```

Construct a fully-specified SQLException
Parameters:
reason - a description of the exception
SQLState - an XOPEN code identifying the exception
vendorCode - a database vendor specific exception code

- **SQLException**

```
public SQLException(String reason,
                    String SQLState)
```

Construct an SQLException with a reason and SQLState; vendorCode defaults to 0.
Parameters:
reason - a description of the exception
SQLState - an XOPEN code identifying the exception

- **SQLException**

```
public SQLException(String reason)
```

Construct an SQLException with a reason; SQLState defaults to null and vendorCode defaults to 0.
Parameters:
reason - a description of the exception

- **SQLException**

```
public SQLException()
```

Construct an SQLException; reason defaults to null, SQLState defaults to null and vendorCode defaults to 0.

Methods

- **getSQLState**

```
public String getSQLState()
```

Get the SQLState
Returns:
the SQLState value

- **getErrorCode**

```
public int getErrorCode()
```

Get the vendor specific exception code
Returns:
the vendor's error code

- **getNextException**

```
public SQLException getNextException()
```

Get the exception chained to this one.
Returns:
the next SQLException in the chain, null if none

- **setNextException**

```
public synchronized void setNextException(SQLException ex)
```

Add an SQLException to the end of the chain.
Parameters:
ex - the new end of the SQLException chain

Class java.sql.SQLWarning

```
java.lang.Object
   |
   +----java.lang.Throwable
           |
           +----java.lang.Exception
                   |
                   +----java.sql.SQLException
                           |
                           +----java.sql.SQLWarning
```

---

public class **SQLWarning**
extends SQLException

The SQLWarning class provides information on a database access warnings. Warnings are silently chained to the object whose method caused it to be reported.

See Also:
getWarnings, getWarnings, getWarnings

Constructor Index

- **SQLWarning**()
  Construct an SQLWarning ; reason defaults to null, SQLState defaults to null and vendorCode defaults to 0.
- **SQLWarning**(String)
  Construct an SQLWarning with a reason; SQLState defaults to null and vendorCode defaults to 0.
- **SQLWarning**(String, String)
  Construct an SQLWarning with a reason and SQLState; vendorCode defaults to 0.
- **SQLWarning**(String, String, int)
  Construct a fully specified SQLWarning.

Method Index

- **getNextWarning**()
  Get the warning chained to this one
- **setNextWarning**(SQLWarning)
  Add an SQLWarning to the end of the chain.

Constructors

SQLWarning

```
public SQLWarning(String reason,
                  String SQLstate,
                  int vendorCode)
```

Construct a fully specified SQLWarning.

Parameters:

reason - a description of the warning
SQLState - an XOPEN code identifying the warning
vendorCode - a database vendor specific warning code

SQLWarning

```
public SQLWarning(String reason,
                  String SQLstate)
```

Construct an SQLWarning with a reason and SQLState; vendorCode defaults to 0.

Parameters:

reason - a description of the warning
SQLState - an XOPEN code identifying the warning

SQLWarning

```
public SQLWarning(String reason)
```

Construct an SQLWarning with a reason; SQLState defaults to null and vendorCode defaults to 0.

Parameters:

reason - a description of the warning

- **SQLWarning**

```
public SQLWarning()
```

Construct an SQLWarning ; reason defaults to null, SQLState defaults to null and vendorCode defaults to 0.

- **getNextWarning**

```
public SQLWarning getNextWarning()
```

Get the warning chained to this one
Returns:
the next SQLException in the chain, null if none

- **setNextWarning**

```
public void setNextWarning(SQLWarning w)
```

Add an SQLWarning to the end of the chain.
Parameters:
w - the new end of the SQLException chain